



WIESŁAW PIETRUSZKIEWICZ

CASCADE STYLE SHEETS

Style Sheets

- At the beginning HTML formatting was done by selected HTML tags
- Cascading Style Sheets (CSS) describe the presentation of a document, created by a markup language – HTML (xHTML)
- CSS stores information about formatting, while HTML stores its content
- Usually it is easier to manage them separately

CSS - Syntax

- Basic syntax:
 - `selector {property: value}`
 - `selector1, selector2 {property1: value1; property2: value2}`
- Selectors may represent different properties used to identify an element or a group of them
- A single element can be identified by many selectors, a selector can identify many elements

Selector – Universal selector

- Selects all elements
- May be restricted to a specific namespace or to all namespaces.

A namespace is an unique information used to remove ambiguity between elements. Rarely used in HTML.

- Syntax:
 - *
 - `specific_namespace|*`
 - `*|*`

Selector – Element's type

- Selects all elements within the document that are of the given type
- Syntax:
 - `input` – matches all input elements
 - `a` – matches all anchors
 - `selected_namespace | p` – all paragraph in a selected namespace

Selector – Element's name

- Selects an element by its id attribute.
- There should be a single element with a given id value i.e. it should be unique property (hence, it has id name)
- Syntax:
 - `#main_anchor` – selects `<a id="main_anchor">`
 - `[id=main_anchor]` – a rarely used equivalent

Selector – Element's class

- Selects elements with class attribute matching the given name
- Class – represents grouped elements (by their meaning, not necessarily by their location)
- Syntax:
 - .external_link – selects
`<a class="external_link" href="http://www.google.com">`
and
`<a class="external_link" href="http://www.bing.com">`
 - [class~="external_links"] – a rarely used equivalent

Selector – Element's attribute

- Selects matching elements i.e. having given attribute or having a given value of selected attribute
- Syntax:
 - `a[title]` – All anchor with a given title attribute
 - `a[href="https://www.google.com"]` – All anchors to `www.google.com`
 - `a[href*="containing_text"]`
 - `a[href$="ending_with"]`
 - `a[href^="starting_with"]`

Selector – Pseudo-classes

- Pseudo-classes specify a special state of selected element
- Denoted by “:” create a dependency on information, not the document tree structure
- Examples:
 - :enabled or :disabled – status of user interface
 - :visited – links that have been visited
 - :playing – currently playing media element

Selector – Pseudo-elements

- Pseudo-elements denoted by “::” specify part of the selected element
- Syntax:
 - ::after – creates an element being the last child of selected element
 - ::first-line – defines the first line of block-level element
 - ::selection – applies style to the part of document selected (highlighted) by user

Selectors - Grouping

- A list of selectors contains a sequence of selectors, separated by “,”
- Use to apply declarations of styles to multiple selectors – no need to rewrite reusable declarations
- Syntax:
 - `p, a {border: blue 1px solid;}`
 - `:is(p, a) {border: blue 1px solid;}` – a rare alternative

Selectors – Combinations

- Combinations define relation between to selectors
- Descendant – represented by a single space
- The first selector relates to an ancestor, while second selector is a descendant
- `li li { /*list item inside list item*/ color: red }`
- Child – represented by `>`
- The second selector is a direct descendant

Selectors – Combinations

- Child – represented by “>”
- The second selector is a direct descendant
- `#header > p { /* nested paragraph(s) in element named header */ margin: 2px; }`
- General sibling – “~” first and second selector must share the same parent
- Adjacent sibling – “+” the second element must immediately follow the first one

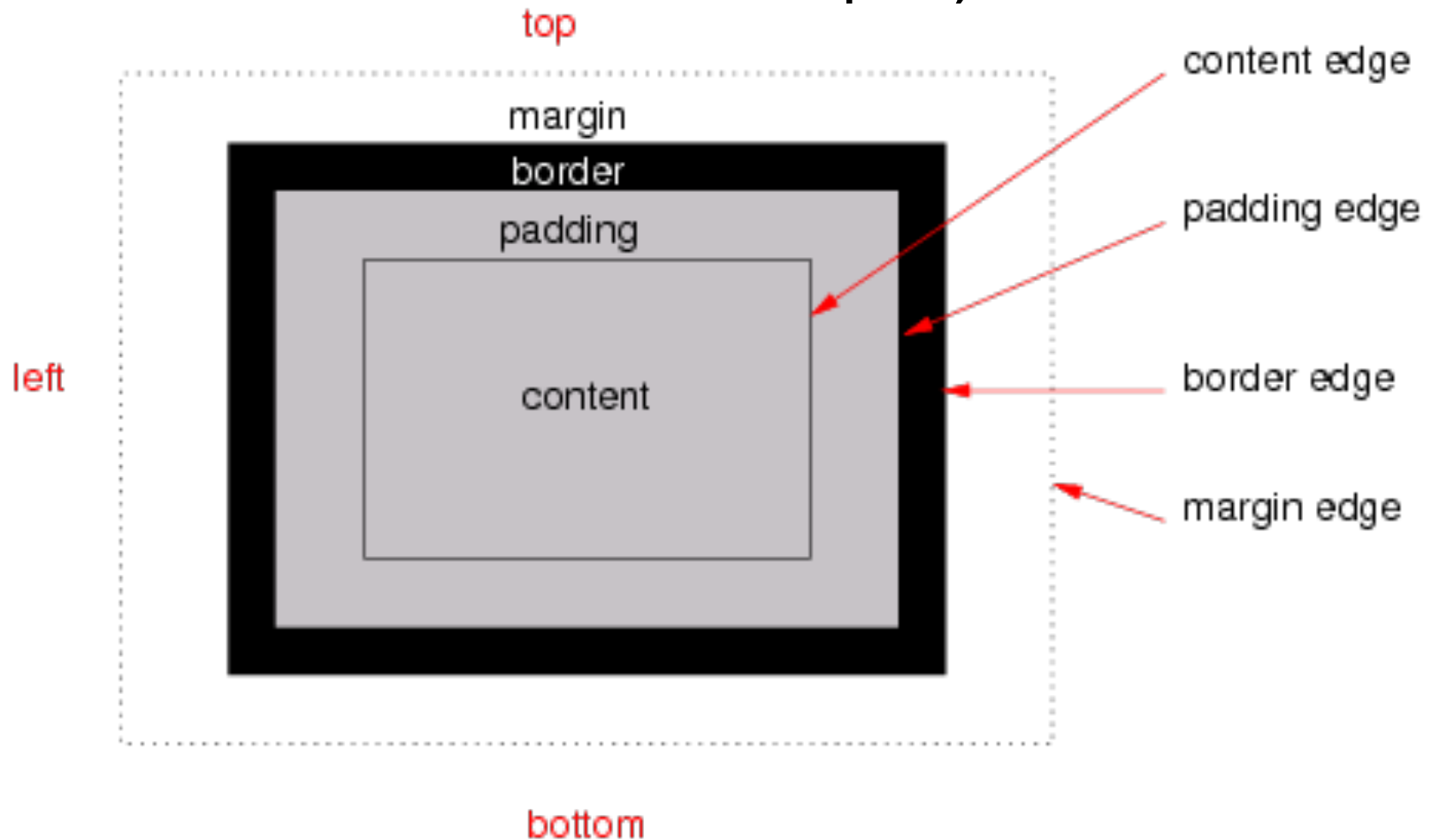
The box model #1

- The key element to understand the placement of elements is to comprehend the box model
- Each element has a content area surrounded by optional:
 - padding – spacing between the content and border;
 - border – visually denoting a padded content;
 - margin – separating element from the other ones.

The box model

CSS Box Model Module Level 3;

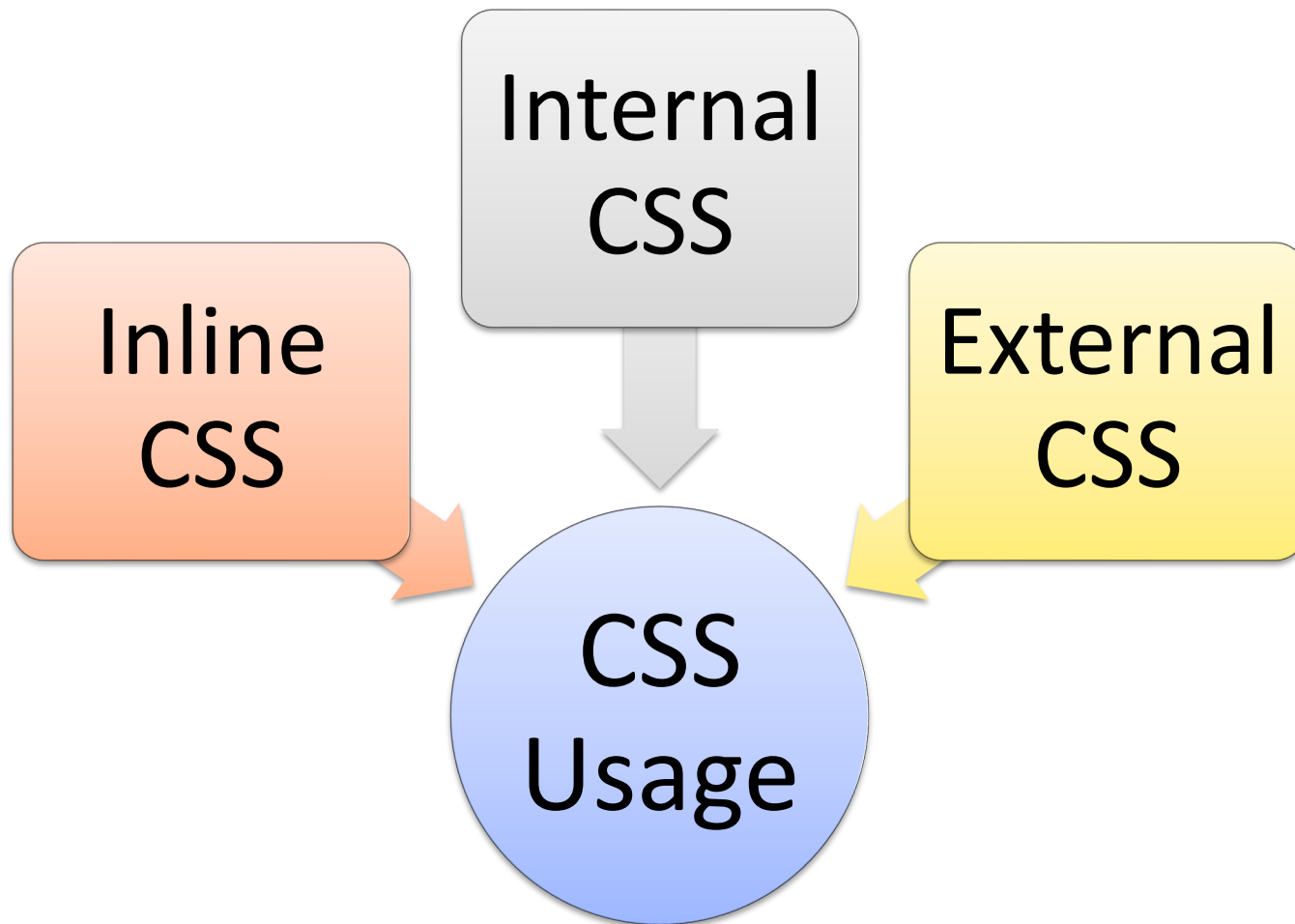
W3C Candidate Recommendation Snapshot, 22 December 2020



CSS – Inheritance

- Inherited properties - property of element is a value computed of that property on its parent element (e.g. colours)
- Non-inherited properties – property that get an initial value of that property (e.g. borders)
- `inherit` – causes element to take property value from its parent
- `all` – sets properties to their initial or inherited values

Ways to use CSS in HTML



Inline CSS

- Applies styles to a single element
- Defines within the element
- Easy to follow information
- Redundant definitions
- No hierarchy in styles
- An example:
 - `<p style="color:red">A red paragraph</p>`

Internal CSS

- Applies styles to a single document
- Written inside the style tag (a part of HTML head section)
- Offers all possibilities of CSS – limited to a single document
- ```
<head>
 <style>
 a {color: blue;}
 </style>
</head>
```

## External CSS

- Applies styles to multiple documents
- Styles are stored in separate file(s)
- Offers all possibilities of CSS – reusable within multiple documents
- CSS files are linked inside head section
- ```
<head>  
<link rel="stylesheet" type="text/css" href="general.css"/>  
<link rel="stylesheet" type="text/css" href="addons.css"/>  
<link rel="stylesheet" type="text/css" href="highcontrast.css"/>  
</head>
```

Styling – Usage scenarios

Inline

- Quick styling
- Focus on styles not hierarchy
- Limited to element & its children
- Isolated styling (no repeated styles to be applied)

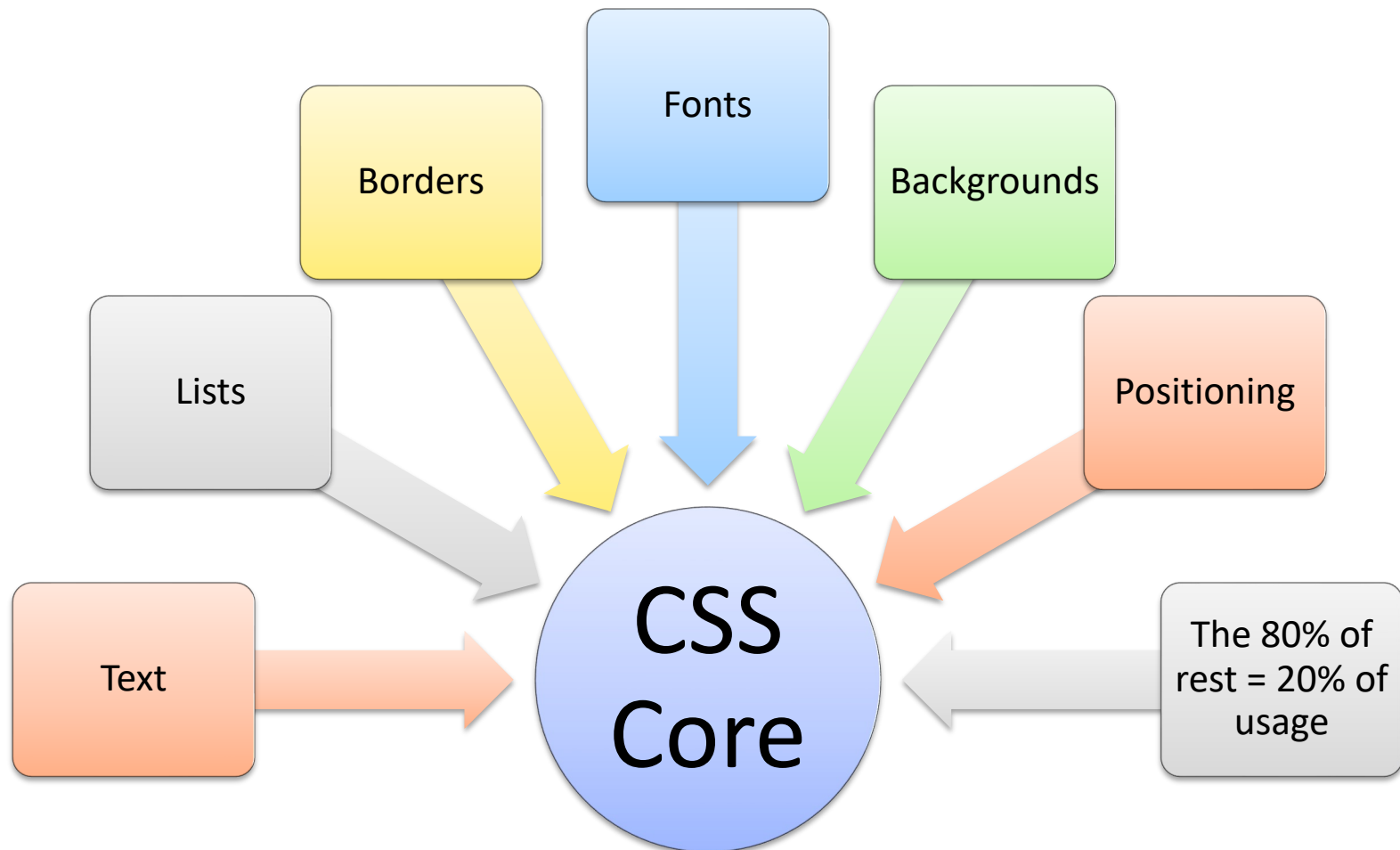
Internal

- Hierarchy of styling
- Optimisation of styling via reusability / common definitions
- HTML & all possibilities of styling in a single file

External

- Internal usage +
- Content separated from styling
- More complex scenarios (multiple files)
- Easier management of multiple separated styling files

CSS – Basic properties



Styles – Sizing

Absolute units

- Absolute units are fixed
- Not recommended for use on screen; Used if the output media is known
 - mm – millimetres
 - cm – centimetres
 - in – inches
 - px – pixels
 - pt – points (1/72 of an inch)
 - pc – picas (1pc = 12pt)

Relative units

- Relative units to another property
- They relate scale to different mediums
 - em – to font-size
 - ex – to x-height of the font
 - ch – to the width of “0”
 - rem – to font-size of the root
 - % - to parent element

Styles – Colours

HSL(A)

- Hue Saturation Lightness Alpha
- Alternative RGB colour model closely aligned with the human perception of colours
- Is a device-independent model
- Examples:
 - `hsla(118, 100%, 50%, 1)`
 - `hsla(11, 100%, 50%, 1)`
 - `hsla(50, 100%, 50%, 1)`
 - `hsla(235, 100%, 50%, 1)`

RGB(A)

- Red Green Blue Alpha
- Additive colour model commonly used in digital vision
- Is a device-dependent model
- Examples:
 - `rgba(8, 255, 0, 1)`
 - `rgba(255, 47, 0, 1)`
 - `rgba(255, 213, 0, 1)`
 - `rgba(0, 21, 255, 1)`

Advanced CSS – Functions

- CSS functions allow to perform various actions on different CSS properties
- They can be considered as a limited substitute of various operations previously done by JavaScript
- The list contains functions frequently used during styling of webpages, that cannot be done using non-functional CSS

CSS Functions – Colours

hsl()

- `hsl(100, 100%, 50%);`

hsla()

- `hsl(100, 100%, 50%, 0.5);`

rgb()

- `color: rgba(255, 0, 255);`

rgba()

- `color: rgba(255, 255, 0, 0.5);`

CSS Functions – Gradients

`conic-gradient()`

- `background-image: conic-gradient(red, yellow, green);`

`linear-gradient()`

- `background-image: linear-gradient(red, blue, yellow);`

`radial-gradient()`

- `background-image: radial-gradient(yellow, green, blue);`

`repeating-conic-gradient()`

- `repeating-conic-gradient(red 10%, yellow 30%);`

`repeating-linear-gradient()`

- `repeating-linear-gradient(red 10%, yellow 30%);`

`repeating-radial-gradient()`

- `repeating-radial-gradient(red 10%, yellow 30%);`

CSS Functions – Filters

NOTE

- All these functions work only with the filter property

brightness()

- filter: brightness(125%);

blur()

- filter: blur(75%);

contrast()

- filter: contrast(200%);

CSS Functions – Misc

attr()

- content: "Broken Image of " attr(alt);

calc()

- calc(120vw - 70px);

cubic-bezier()

- cubic-bezier(0.14, 0.65, 0.81, 0.64);

max() and min()

- width: max(400px, 45%);
height: min(300px, 50%);

url()

- background: url(/images/background.png);

var()

- html { --colour: red; }
a { color: var(--colour); }

CSS Functions – Transformation

scaleX(), scaleY(), scaleZ(), scale3d(), and scale()

•transform: scale(2.5);

**translateX(), translateY(), translateZ(), translate3d(),
and translate()**

•.centered { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }

rotateX(), rotateY(), rotateZ(), rotate3d(), and rotate()

•transform: rotate(25deg);

grayscale()

•filter: grayscale(75%);

invert()

•filter: invert(100%);

perspective()

•transform: perspective(100em) rotateX(45deg);

opacity()

•filter: opacity(10%);

skewX(), skewY(), and skew()

•transform: skew(35deg, 25deg);

saturate()

•filter: saturate(50%);

CSS - References

- Detailed list of selectors:

https://developer.mozilla.org/en-US/docs/Web/CSS/Attribute_selectors

https://developer.mozilla.org/en-US/docs/Web/CSS/Class_selectors

https://developer.mozilla.org/en-US/docs/Web/CSS/ID_selectors

https://developer.mozilla.org/en-US/docs/Web/CSS/Nesting_selector

https://developer.mozilla.org/en-US/docs/Web/CSS/Type_selectors

https://developer.mozilla.org/en-US/docs/Web/CSS/Universal_selectors

- Detailed list of properties:

<https://developer.mozilla.org/en-US/docs/Web/CSS>

HTML formatting vs CSS – A selection

Format	HTML	CSS
Bold text	<code></code>	<code>font-weight: bold;</code>
Important text	<code></code>	<code>font-weight: bold;</code>
Italic text	<code><i></code>	<code>font-style: italic;</code>
Emphasized text	<code></code>	<code>font-style: italic;</code>
Marked text	<code><mark></code>	<code>background-color: yellow; color: black;</code>
Smaller text	<code><small></code>	<code>font-size: smaller;</code>
Deleted text	<code></code>	<code>text-decoration: line-through;</code>
Inserted text	<code><ins></code>	<code>text-decoration: underline;</code>
Subscripted text	<code><sub></code>	<code>vertical-align: sub; font-size: smaller;</code>
Superscripted text	<code><sup></code>	<code>vertical-align: super; font-size: smaller;</code>
Font style	<code></code> Not in HTML5	Misc. properties (family, size, stretch, style, variant, weight or line-height)
Element's width	width attribute	<code>width: 50%;</code>
Element's height	height attribute	<code>height: 20em;</code>

HTML formatting vs CSS - Usage

HTML

- + Simpler technology stack
- + Quicker to deploy (for small modifications)
- + Done on-spot
- + Effects clear for a person reading source code
- Simple possibilities
- Redundant
- Mixes content with formatting

CSS

- + A natural choice in the community
- + Separated content from its formatting
- + Supporting editing tools
- + More possibilities
- + Reusable elements
- More complex dependencies between elements
- Higher skills required

CSS Animations

- It is possible to create animations without JavaScript via CSS
- Animation – gradually changes from one style to another
- Key element – a keyframe defining properties to to be used as an reference values (being a starting, ending or transitional point)

CSS Animations – MDN Example

```
CSS Play 
```

```
:root {  
  overflow: hidden; /* hides any part of the sun below the horizon */  
  background-color: lightblue;  
  display: flex;  
  justify-content: center; /* centers the sun in the background */  
}  
  
.sun {  
  background-color: yellow;  
  border-radius: 50%; /* creates a circular background */  
  height: 100vh; /* makes the sun the size of the viewport */  
  aspect-ratio: 1 / 1;  
  animation: 4s linear 0s infinite alternate sun-rise;  
}  
  
@keyframes sun-rise {  
  from {  
    /* pushes the sun down past the viewport */  
    transform: translateY(110vh);  
  }  
  to {  
    /* returns the sun to its default position */  
    transform: translateY(0);  
  }  
}
```

CSS Animations – MDN Example

HTML

Play 



```
<div class="sun"></div>
```

Play 



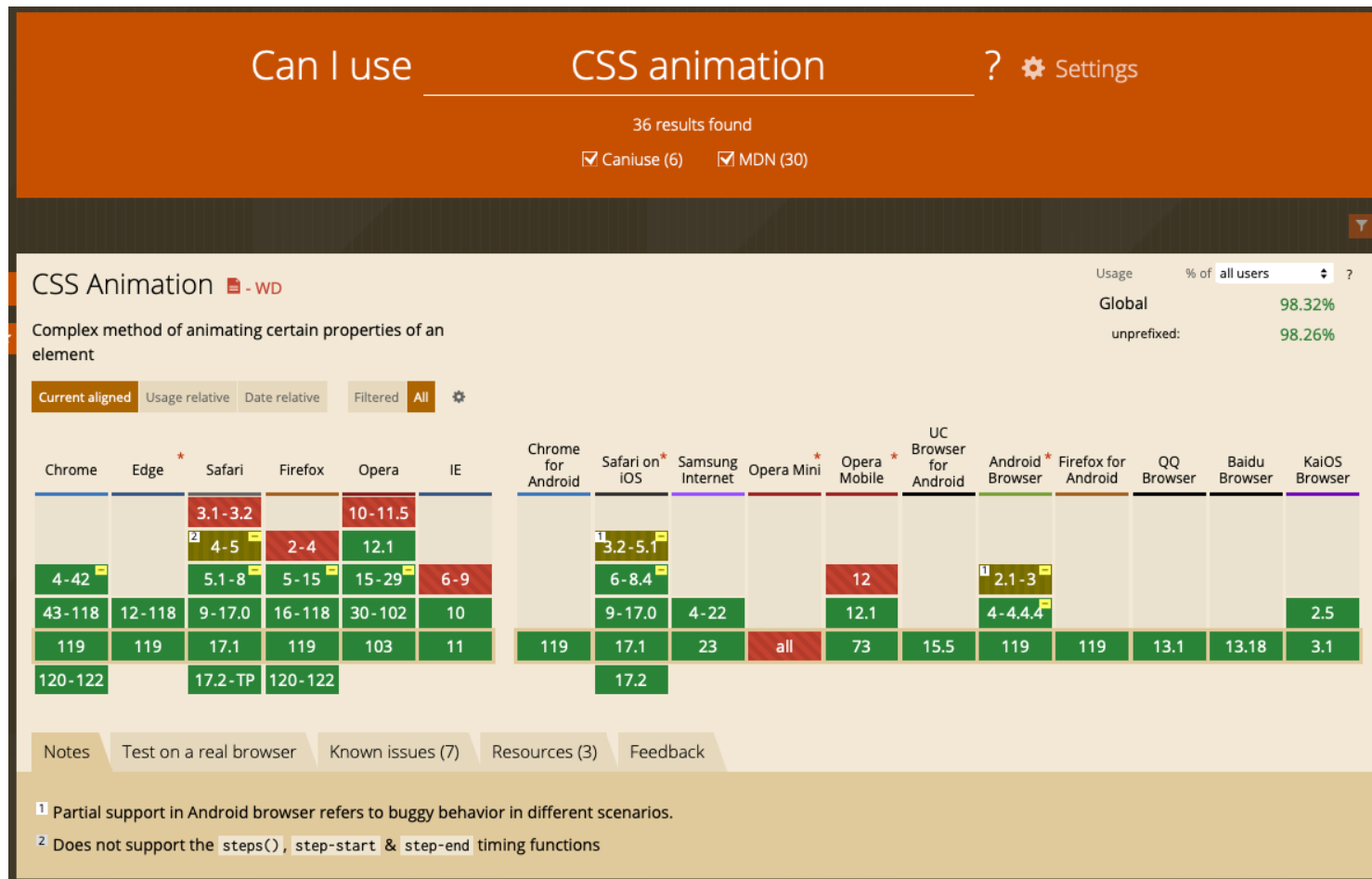
CSS Animations – Libraries

- To make the process easier there are libraries available, they contain common elements
- For a review of CSS animation libraries take a look at:

<https://levelup.gitconnected.com/12-best-css-animation-libraries-7d1b0848dfca>

<https://freefrontend.com/css-animation-libraries/>
<https://coderpad.io/blog/development/top-10-css-animation-libraries/>

CSS Animation – Usage



Media queries

- Media query allow us to apply styles depending of device's type or viewport width
- Common usage scenarios – differentiate styles for screen or printing
- Examples (from MDN):

CSS

```
@media screen and (min-width: 30em) and (orientation: landscape) {  
    /* ... */  
}
```

CSS

```
@media (min-height: 680px), screen and (orientation: portrait) {  
    /* ... */  
}
```


CSS – Common elements

Create delightful login experiences in minutes with a no-code workflow builder

Keep Calm and Sign On

Don't want to see ads?

mdn web docs

References Guides Plus Blog Play AI Help

Theme

Log in

Sign up for free

References > CSS > CSS Layout cookbook

Filter

CSS

Tutorials

CSS basics

CSS first steps

CSS building blocks

Styling text

CSS layout

Reference

Modules

Properties

Selectors

Combinators

Pseudo-classes

Pseudo-elements

At-rules

Functions

Types

Guides

Animations

Backgrounds and Borders

Box alignment

Box model

Colors

Columns

CSS Layout cookbook

The CSS layout cookbook aims to bring together recipes for common layout patterns, things you might need to implement in your own sites. In addition to providing code you can use as a starting point in your projects, these recipes highlight the different ways layout specifications can be used, and the choices you can make as a developer.

Note: If you are new to CSS layout then you might first like to take a look at our [CSS layout learning module](#), as this will give you the basic grounding you need to make use of the recipes here.

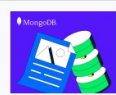
The Recipes

Recipe	Description	Layout Methods
Media objects	A two-column box with an image on one side and descriptive text on the other, e.g. a facebook post or tweet.	CSS Grid , float fallback, fit-content sizing
Columns	When to choose multi-column layout, flexbox or grid for your columns.	CSS Grid , Multicol , Flexbox
Center an element	How to center an item horizontally and vertically.	Flexbox , Box Alignment
Sticky footers	Creating a footer which sits at the bottom of the container or viewport when the content is shorter.	CSS Grid , Flexbox
Split navigation	A navigation pattern where some links are visually separated from the others.	Flexbox , margin
Breadcrumb navigation	Creating a list of links to allow the visitor to navigate back up through the page hierarchy.	Flexbox

In this article

The Recipes

Contribute a Recipe



Imagine what you'll build tomorrow

Don't want to see ads?

4. Cascade Style Sheets — Saved to my Mac

https://developer.mozilla.org/en-US/docs/Web/CSS/Layout_cookbook

EXTENSIONS TO CSS

CSS Preprocessors

- To make CSS development easier there exists a range of CSS preprocessors
- They translate styles from one language or a superset of CSS into a standard CSS
- Translated language should improve the performance of styling development
- There are a few of CSS preprocessors – differing in the approach to this task
- Additional complexity – An addon to HTML+JS+CSS

LESS

- Dynamic preprocessor based on Less language (a superset of CSS)
- To be run on server or client side (real time compilation via less.js)
- Usage of variables
- Nested styles
- Mixins – functions returning styles
- Additional functions

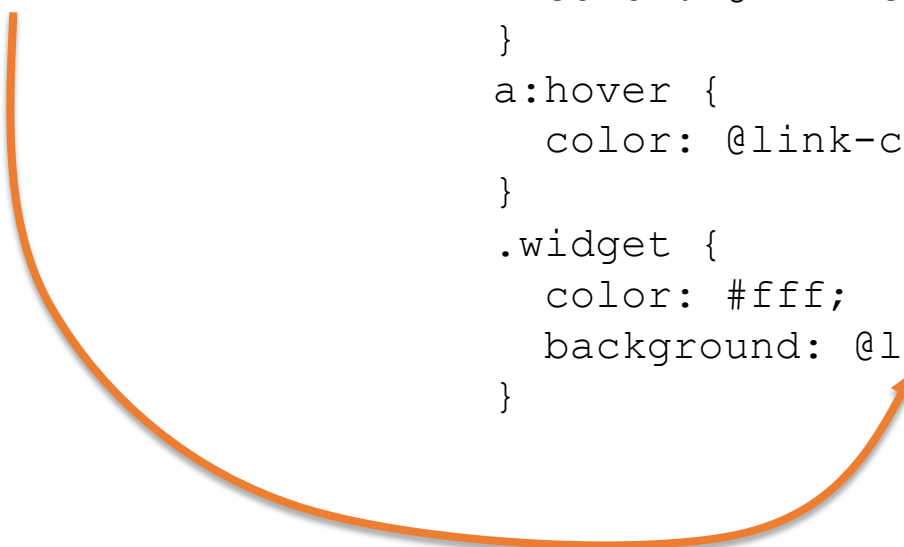
LESS – Example (lesscss.org)

```
a,
.link {
  color: #428bca;
}

.widget {
  color: #fff;
  background: #428bca;
}

// Variables
@link-color:          #428bca; // sea blue
@link-color-hover:    darken(@link-color, 10%);

// Usage
a,
.link {
  color: @link-color;
}
a:hover {
  color: @link-color-hover;
}
.widget {
  color: #fff;
  background: @link-color;
}
```



Stylus

- A “pythonic” approach to CSS (indent-based)
- Build for node.js – compiled on server
- Dynamic styles (e.g. iterations - for loops)
- Mixins & many build-in functions
- Conditionals (if-else if – else)

Stylus - Example

```
fonts = Helvetica, Arial, sans-serif
body {
  padding: 50px;
  font: 14px/1.4 fonts;
}
```



```
body {
  font: 12px Helvetica, Arial, sans-serif;
}
a.button {
  -webkit-border-radius: 5px;
  -moz-border-radius: 5px;
  border-radius: 5px;
}
```

Sass

- Another CSS preprocessor
- Variables ...
- Nesting ...
- Partials + Modules – modularization of CSS
- Mixins ...
- Extend/inheritance – CSS property sharing

Sass - Example

```
nav
  ul
    margin: 0
    padding: 0
    list-style: none
  li
    display: inline-block
  a
    display: block
    padding: 6px 12px
    text-decoration: none

nav ul {
  margin: 0;
  padding: 0;
  list-style: none;
}
nav li {
  display: inline-block;
}
nav a {
  display: block;
  padding: 6px 12px;
  text-decoration: none;
}
```



CSS-in-JS

- Another approach to styling, where CSS is generated by JavaScript
- CSS is later attached into the DOM
- Uses flexibility and power of JavaScript (exceeding the power of CSS preprocessors)
- JavaScript isn't a domain-specific language (comparing to CSS preprocessors)
- Multiple additional JS libraries

CSS-in-JS - Example (styled-components)

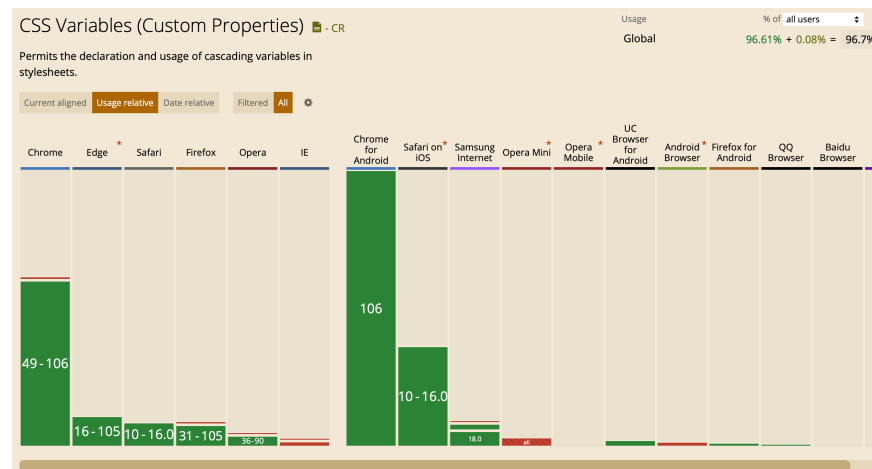
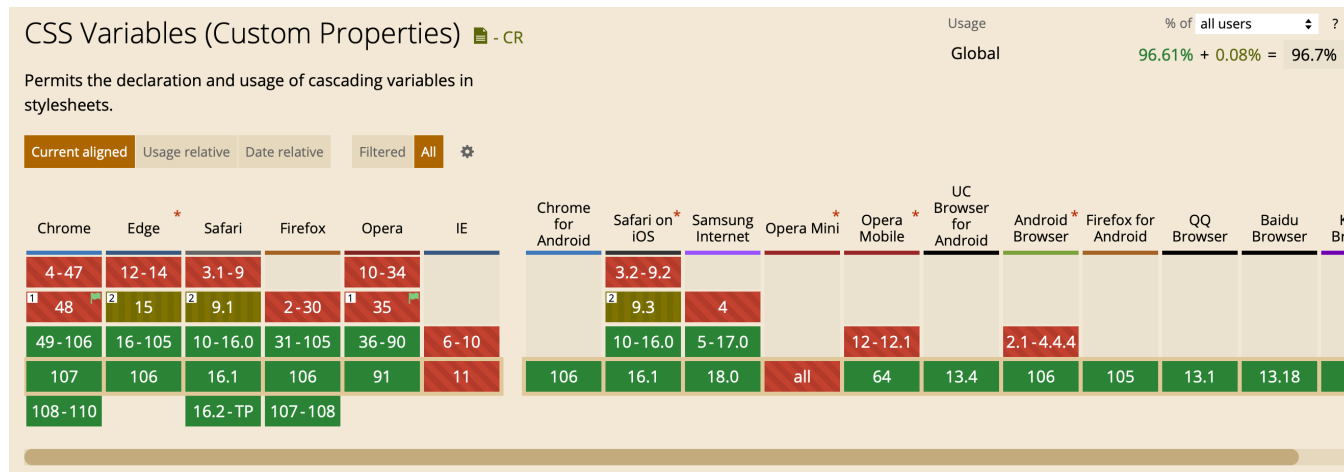
```
const Button = styled.button`
  background: transparent;
  border-radius: 3px;
  border: 2px solid palevioletred;
  color: palevioletred;
  margin: 0.5em 1em;
  padding: 0.25em 1em;

  ${props => props.primary && css`
    background: palevioletred;
    color: white;
  `}
`;

const Container = styled.div`
  text-align: center;
`;

render(
  <Container>
    <Button>Normal Button</Button>
    <Button primary>Primary Button</Button>
  </Container>
);
```

Support for CSS variables (caniuse.com)



Summary

- CSS isn't an additional element to web styling
- CSS is an essential piece of the web core
- HTML styling offers only a very limited subset of CSS capabilities
- There are CSS preprocessors available, enhancing CSS or making development easier
- CSS is updated by an introduction of additional elements (e.g. functions)

TILL NEXT LECTURE